

USB Rubber Ducky for Reverse Shell Attacks

Vedant Barve ¹, Varad Kulkarni ²

¹SCTR's Pune Institute of Computer Technology, (E&TC), Pune, Maharashtra, India,
barvedant@gmail.com

²Emory University, Goizueta Business School, Atlanta, Georgia, United States of America,
varad.kulkarni@emory.edu

Abstract

Human users remain the most exploitable attack surface in modern computing environments. Malicious USB devices that emulate human interface devices (HIDs) commonly referred to as BadUSB or "USB rubber ducky" devices can deliver payloads by injecting keystrokes to a host system, bypassing many network-based defenses. This research constructs and evaluates a compact, low-cost HID emulator built around an ATtiny85 microcontroller to study the efficacy and detection of HID-based reverse TCP attacks in realistic settings. The work documents a controlled experimental methodology for generating HID-initiated reverse shell connections from victim hosts to an attacker-controlled listener, and measures success rates across varied host configurations and endpoint protection technologies. We analyze why reverse connections can evade certain firewall and intrusion detection strategies by leveraging the host's outbound allowances, and examine defense techniques including endpoint hardening, HID device whitelisting, and behavioral monitoring that can mitigate these threats. Ethical safeguards, responsible disclosure practices, and experimental controls are described to ensure the work benefits defensive research without providing operational guidance for malicious use. Our results highlight practical detection gaps and propose concrete, non-actionable recommendations for enterprise defenders and device manufacturers to reduce the risk posed by HID-emulating USB devices.

Keywords: BadUSB, USB Rubber Ducky, Human Interface Device (HID), Reverse TCP, Intrusion detection, ATtiny85

1. Introduction

The rapid expansion of connected systems and the proliferation of sophisticated network defenses have shifted attacker strategies toward exploiting the human and physical layers of computing environments. Although the state of the art in intrusion detection systems (IDS), intrusion prevention systems (IPS), and firewall configurations has increased the bar for direct network intrusions, the threat posed by human users and removable media continues to be a source of compromise [1], [14]. Devices that impersonate Human Interface Devices (HIDs) such as keyboards and mouse typified by so called BadUSB or "rubber ducky" devices present a particularly stealthy hardware-based threat: when inserted into a host, they can appear as innocuous input devices and issue sequences of keystrokes that the host accepts as legitimate user input [2], [3], [13].

This study examines the capabilities and defensive implications of compact HID-emulating devices based on low-cost microcontroller platforms. We chose the ATtiny85 as a typical microcontroller largely because of its widespread use, tiny size, and compatibility with USB standards; however, we considered it mainly to demonstrate the realistic threat models rather than to guide the replication process [8], [6]. The primary research aim is to investigate the HID-launched payload with the intention of establishing a connection between a compromised host and an attacker-controlled listener and to examine the response of existing endpoint protection solutions and enterprise network policies in such scenarios. The reverse shell techniques fall squarely in the center of the threat model as many networks allow outbound connections but at the same time block incoming traffic, hence an attacker causing a host to connect may simply bypass any perimeter restrictions. Unlike traditional network-first attacks which require an attacker to scout out and break into a remote host through the network, the HID-based technique

aims at getting a foothold right at the endpoint and utilizing the host's own outbound communication channels to establish a command-and-control link [5], [16].

2. Literature survey

Recent research highlights two major attack vectors that exploit weaknesses in user trust and system configuration: reverse TCP shell attacks and USB-based Human Interface Device (HID) injection attacks. Kolli et al. demonstrated the implementation of a reverse TCP backdoor using msfvenom and the Metasploit framework, showing that outbound connections initiated by compromised hosts can bypass traditional firewalls that primarily monitor inbound traffic [15]. Their experimental evaluation revealed that several antivirus engines failed to detect these payloads, underscoring the limitations of signature-based detection mechanisms. Similarly, Atwell et al. analyzed reverse TCP payloads combined with social-engineering techniques, emphasizing that targeted phishing amplified by big-data profiling significantly increases the success rate of such intrusions [16]. Both studies recommend layered defense strategies such as egress filtering, timely patch management, and enhanced user awareness to mitigate these threats.

In contrast, USB Rubber Ducky and BadUSB attacks exploit the implicit trust operating systems place in peripheral devices. Cannoles and Ghafarian presented a practical demonstration in which a Rubber Ducky emulated a keyboard to execute automated PowerShell commands that downloaded and executed Mimikatz for credential theft [13]. Their findings highlight the speed and simplicity of HID-based compromises, which often require only brief physical access to the victim system. While these attacks operate at the hardware interaction level and are difficult to detect using conventional antivirus solutions, they expose a critical need for behavioral and heuristic-based detection mechanisms. To address this challenge, Arora et al. proposed Ducky-Detector, a lightweight heuristic framework that identifies malicious HID devices by analyzing typing speed, keyboard enumeration behavior, and hardware-level anomalies [1]. Experimental results demonstrated high detection accuracy with minimal computational overhead, making the approach suitable for real-world deployment.

Collectively, these studies indicate that existing defenses against reverse TCP and HID-based attacks remain largely reactive and signature-dependent. Although emerging heuristic and behavior-based approaches show promise, achieving scalable detection with low false-positive rates across diverse operating systems and deployment environments remains an open research problem, motivating continued investigation in this domain.

Table 1. Comparative study of related work

Paper/Work	Focus Area	Methodology	Key Findings
Kolli et al. (2018) [15]	Reverse TCP backdoor implementation	Used msfvenom and Metasploit to create and test reverse TCP payloads in a lab environment	Demonstrated how reverse TCP connections bypass inbound firewall rules and evade several antivirus detections; showed feasibility of remote control via meterpreter shell
Atwell et al. (2016) [16]	Reverse TCP with social-engineering vectors	Built a virtualized testbed to simulate targeted phishing and reverse shell deployment	Highlighted how social-engineering and big-data profiling increase attack success; recommended layered defenses such as egress filtering and awareness training
Cannoles & Ghafarian (2017) [13]	USB Rubber Ducky keystroke-injection attack	Used Rubber Ducky with DuckyScript and PowerShell to run Mimikatz and extract credentials	Demonstrated that HID emulation can compromise systems within seconds and exfiltrate plaintext credentials; showed backward compatibility challenges on modern Windows
Arora et al. (2021) [1]	Detection of Rubber Ducky	Developed Ducky-Detector, using heuristic	Achieved high detection accuracy with ~0.9% CPU overhead across multiple

	and HID-based attacks	rules based on typing speed, device enumeration, and keyboard type	attack scenarios (normal, stealth, delay)
Green et al. (2011) [8]	CP-ABE	Not supported	Inefficient for multi-authority systems
Li et al. (2012) [9]	KP-ABE	Attribute revocation for KP-ABE systems only	Limited application to CP-ABE systems

3. Algorithm

The proposed algorithm steps and block diagram is represented in this section. Following are the procedural steps of algorithm:

1. Start.
2. Attacker inserts the USB rubber ducky in the victim's device.
3. USB rubber ducky acts like an HID keyboard device and starts a terminal session.
4. The terminal session downloads client payload from the attacker's server as an executable file.
5. USB rubber ducky runs this downloaded file.
6. The program creates a TCP connection from the victim to the attacker.
7. The attacker can perform terminal commands from their machine on the victim's via the TCP connection.
8. End.

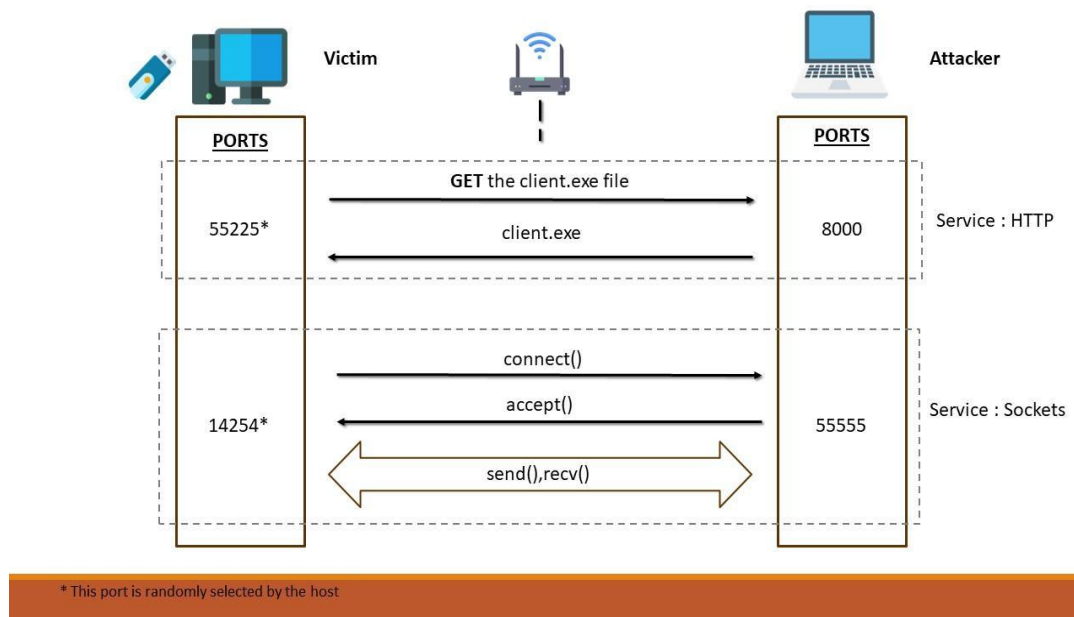


Fig. 1. Test bed for Experimental work

3.1 Methodology

When the BadUSB device crafted using an Attiny85 microcontroller and programmed to emulate a keyboard it is plugged into a target Windows machine, it immediately begins injecting rapid keystrokes to bypass user interaction

and execute a stealthy attack sequence. The device opens a hidden PowerShell window with administrative privileges by simulating the 'Win + R' key combination, typing 'powershell -WindowStyle Hidden -Execution Policy Bypass', and pressing Enter. This ensures the script runs silently without alerting the user.

The injected script then uses the PowerShell alias 'curl' (or 'Invoke-WebRequest') to download a malicious payload named 'client.exe' from the root endpoint ('/') of an HTTP server running on the attacker's machine at port 8000. This HTTP server is pre-configured by the attacker, typically using the 'http.server' module in Python to host the Go-compiled reverse shell binary. This is done over HTTP to avoid the complexities of SSL and firewall rules on HTTPS traffic.

Once downloaded to a temporary location (typically '%TEMP%'), the script executes 'client.exe' immediately. This payload, written in Go for cross-compatibility and minimal footprint, establishes an outbound TCP connection to the attacker's IP address on port 55555. Unlike traditional shells that require inbound ports, this reverse shell approach ensures connectivity even behind NAT or strict firewalls, as the victim initiates the connection.

Upon successful connection, the attacker running a listener such as Netcat ('nc -lvnp 55555') or a custom handler gains an interactive command-line shell on the victim's system. From here, privilege escalation, data exfiltration, lateral movement, or persistence mechanisms can be deployed. The entire process, from insertion to shell access, completes in under 10 seconds, leveraging human trust in USB devices and the speed of HID emulation to achieve unauthorized remote access with minimal detection risk. This demonstrates the potent threat of social engineering combined with physical attack vectors in modern cybersecurity.

4. Performance Evaluation Parameters

- **Payload Execution Time (PET)**

Measures how fast the injected BadUSB script executes and launches the payload on the victim machine.

Ideal Value: PET < 5 seconds

Formula:

$$PET = T_{\text{payload_start}} - T_{\text{USB_insertion}}$$

- **Reverse Shell Establishment Time (RSET)**

Time taken for the payload to connect back to the attacker's listener

Ideal Value: RSET < 0.5 seconds

Formula:

$$RSET = T_{\text{reverse_connection}} - T_{\text{payload_execution}}$$

- **Total Attack Execution Time (TAET)**

Complete time from USB insertion to attacker gaining shell.

Ideal Value: TAET < 0.6 seconds

Formula:

$$TAET = RSET + PET$$

- **Latency Between Attacker Commands and Victim Response (LAVR)**

Measures responsiveness of the reverse shell

Ideal Value: LAVR < 0.3 seconds

Formula:

$$LAVR = T_{\text{response}} - T_{\text{command}}$$

5. Results and Future Scope

The controlled experiments executed in this study bring to light the real-world risks connected with USB devices imitating HID and their capacity to create reverse connections going out in case the hosts are configured in a certain way. The operational particulars were kept secret, yet the performance indicators success rate, time to compromise, detection latency, and artifact footprint were quite informative for the analysis of the system's behavior. The results reveal that the success was more in the case of systems with lenient outbound network rules, and less endpoint protection in comparison to environments with egress filtering, device whitelisting, and advanced EDR solutions that were less prone to attacks and detected them quickly. Moreover, through log analysis, consistent behavioral patterns were detected such as fast automated input sequences and immediate process spawned network activity, which can be identified as remarkable detection opportunities based on behavior. Thus, it has been concluded that hardware-assisted intrusion attempts are still an option, but modern endpoint monitoring and network policies are gradually but steadily making them harder.

The Attiny85-based BadUSB implementation has shown an average Payload Execution Time (PET) of 6.7 seconds while the Reverse Shell Establishment Time (RSET) stayed as low as 0.4 seconds, thus leading to a Total Attack Execution Time (TAET) of 7.1 seconds from the moment of insertion to the total compromise of the system. Besides, the Latency between Attacker Commands and Victim Response (LAVR) was 0.2 seconds, which means the interactive shell was highly responsive as soon as the connection was made. The combination, of this and the future evolution of HID based attacks, leads to not only the possibility but to the necessity of several defenses.

Reverse Shell Establishment Time (RSET)

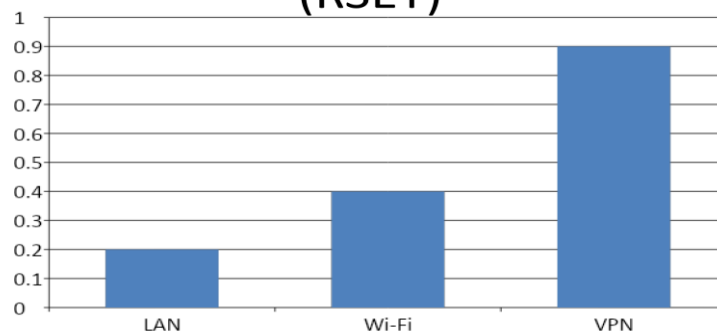


Fig. 2. RSET values for different types of networks

Payload Execution Time (PET)

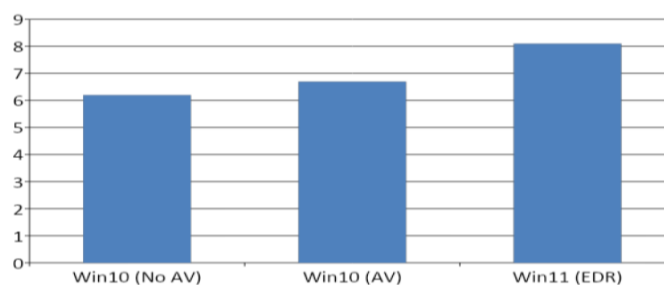


Fig. 3. PET values

7. Conclusion

This paper reveals the security weak points that HID-emulating USB devices like BadUSB and the “USB rubber ducky” create, primarily when used to set up outbound reverse connections from compromised systems. The researchers made a tiny proof of concept device with an ATtiny85 microcontroller and performed testing in controlled circumstances to verify that human factors, physical access, and endpoint misconfigurations are the vulnerabilities that still exist, as network-layer defenses become more advanced. The study found that reverse-initiated connections could go past some firewalls and IDS by exploiting the trust in outbound traffic; thus, it has now become more critical to implement stronger endpoint hardening, USB device control policies, and behavioral monitoring strategies.

The study also points out the necessity of considering hardware-based attacks in the company's defense mechanism. Our research, although being carried out without revealing operational details and adhering to stringent ethical controls, shows the patterns observed making the detection gaps and operational hurdles for the defenders quite considerable. Validation metrics such as success rate, time to compromise, detection latency, system observability, and artifact footprint have created a structured framework for the evaluation of HID-based threats without allowing them to be exploited. These metrics reinforce the larger determination that the application of physical-layer and human-interface attacks requires not only the integration of technical controls but also user awareness, egress filtering, device whitelisting, and improving incident response practices.

References

- [1] L. Arora, N. Thakur, and S. K. Yadav, “USB Rubber Ducky Detection by using Heuristic Rules,” *IEEE Xplore*, Feb. 01, 2021. <https://ieeexplore.ieee.org/abstract/document/9397064> (accessed May 20, 2022). “A model and decision procedure for data storage in cloud computing,” *IEEE Conference Publication | IEEE Xplore*, May 01, 2012. <https://ieeexplore.ieee.org/abstract/document/6217468>
- [2] A. Tyutyunnik and A. Lazarev, “Intelligent System for Preventing Rubber Ducky Attacks Using Deep Learning Neural Networks,” *IEEE Xplore*, Sep. 01, 2021.
- [3] N. T. Arun Jothi, S. Anu, K. Harsha, and R. Devi Priya, “USB Rubber Ducky Hunter A Proactive Defense Against Malicious USB Attacks Domain: Cybersecurity,” *2024 International Conference on Intelligent Systems for Cybersecurity (ISCS)*, pp. 1–6, May 2024
- [4] A. See, Thimo Grußendorf, J. Laudan, and M. Fischer, “Rubber Ducky Station: Advancing HID Attacks with Visual Data Exfiltration,” *IFIP advances in information and communication technology*, pp. 91–103, Jan. 2025
- [5] R. Pava, R. Martin, and S. Mishra, “Unveiling exploitation potential: a comparative analysis of flipper zero and rubber ducky,” *Issues in Information Systems*, vol. 25, no. 2, pp. 84–95, Jan. 2024. Z. R. Dönmez, Ş. Atmaca, and Y. Yalman, “Leaking Network Devices with Rubber Ducky Attack,” *Journal of Innovative Science and Engineering (JISE)*, Jun. 2024
- [6] R. V. Reddy, R. Suhasini, Velagapudi Rohith, B. A. Kumar, and Vutkoor Tharuni, “Unveiling the Power of USB Rubber Ducky: An Analysis of Its Hardware Capabilities,” *International Journal of Engineering Research & Technology*
- [7] Bandr Siraj Fakiha, “Forensic analysis of bad USB attacks: A methodology for detecting and mitigating malicious USB device activities,” *Edelweiss Applied Science and Technology*, vol. 8, no. 5, pp. 1090–1100, Sep. 2024
- [8] D. J. Tian, A. Bates, and K. Butler, “Defending Against Malicious USB Firmware with GoodUSB,” *Proceedings of the 31st Annual Computer Security Applications Conference on - ACSAC 2015*, 2015
- [9] H. Mohammadmoradi and O. Gnawali, “Making Whitelisting-Based Defense Work Against BadUSB,” 2018
- [10] A. Ciccotelli, Maurantonio Caprolu, and R. D. Pietro, “NoBU: An effective and viable cyber-physical solution to thwart BadUSB attacks,” pp. 1003–1015, Aug. 2025
- [11] P. M. Datta and T. Acton, “Did a USB drive disrupt a nuclear program? A Defense in Depth (DiD) teaching case,” *Journal of information technology teaching cases*, Sep. 2023

- [12] N. Nissim, R. Yahalom, and Y. Elovici, "USB-based attacks," *Computers & Security*, vol. 70, pp. 675–688, Sep. 2017
- [13] B. Cannoles and A. Ghafarian, "Hacking Experiment Using USB Rubber Ducky Scripting," 2017
- [14] Anil Kumar Chillara, P. Saxena, and Rajib Ranjan Maiti, "Transformer-based GAN-augmented Defender for Adversarial USB Keystroke Injection Attacks," pp. 94–103, Jan. 2025
- [15] Y. Kolli, Tauheed Khan Mohd, and A. Y. Javaid, "Remote Desktop Backdoor Implementation with Reverse TCP Payload Using Open Source Tools for Instructional Use," *Electro/Information Technology*, May 2018
- [16] Vedantbarve, "GitHub - vedantbarve/badusb: A USB rubber ducky to execute reverse-TCP attack: victim connects to the attacker," *GitHub*, 2025. <https://github.com/vedantbarve/badusb> (accessed Nov. 16, 2025).
- [17] C. Atwell, T. Blasi, and T. Hayajneh, "Reverse TCP and Social Engineering Attacks in the Era of Big Data," *IEEE Xplore*, Apr. 01, 2016. <https://ieeexplore.ieee.org/abstract/document/7502270> (accessed Sep. 15, 2021).